

Part 1 – Autonomous Systems Fundamentals

Ziad Tarek Fouad

Contents

1	General	2
1.1	Question 1	2
1.2	Question 2	2
1.3	Question 3	3
1.4	Question 4	3
2	ROS2	4
2.1	Question 1	4
2.2	Question 2	4
3	Path Planning	5
3.1	Question 1	5
3.2	Question 2	5
3.3	Q3: Occupancy Grid vs. Height Maps	6
3.4	Question 4	6
3.5	Q5: Artificial Potential Field (APF)	7
4	Control	7
4.1	Q1: Control Theory	7
4.2	Q2: Open-Loop vs. Closed-Loop	7
4.3	Q3: Controller Stability	7
4.4	Q4: Pure Pursuit	7
4.5	Q5: Adaptive Pure Pursuit	8
5	Localization	8
5.1	Q1: Sensor Fusion	8
5.2	Q2: IMU Function and Application	8
5.3	Q3: Kalman Filter (KF) vs. Extended Kalman Filter (EKF)	8
6	Perception	8
6.1	Q1: LiDAR vs. Camera	8
6.2	Q2: Monocular vs. Stereo Cameras	8
6.3	Q3: ArUco Markers	9
7	Conclusion	9

1 General

1.1 Question 1

An autonomous system is a system that perceives its environment, makes decisions, and acts on its own to achieve a goal without continuous human intervention. It operates in a closed loop: **sense** → **plan** → **act** → **repeat**.

- **Perception Module:** This module collects raw data from sensors (LiDAR, Cameras, IMU) and extracts meaningful information about the surrounding environment. **Raw data includes:** Object detection, classification (pedestrians, vehicles, traffic signs), semantic segmentation (identifying road lanes vs. curbs), and tracking of dynamic obstacles. **Output will be:** obstacles, landmarks, free space/occupancy grid.
- **Localization Module:** Localization determines the precise position (pose: X, Y, Z, orientation) of the vehicle with respect to a global map like HD map, using sensors like GNSS (Global Navigation Satellite System), IMU (inertial motion unit), wheel odometry, and LiDAR/Camera matching against a pre-built map to achieve centimeter-level accuracy. Takes input from perception's output in addition to sensors data to **output robot pose (position + orientation)**.
- **Planning Module:** This module takes input from perception and localization to make strategic and tactical decisions to reach a goal safely. It consists of three types:
 - **Behavioral Planning:** Decision-making, such as overtaking, stopping at lights, or yielding.
 - **Motion/Path Planning:** Generating a safe, collision-free, and smooth path for the vehicle to follow.
 - **Trajectory planning:** To indicate obstacles and objects and plan a path away from them to avoid crossing them.

It takes input from localization module (position) and map and goal. **Output:** A planned path (sequence of waypoints, speed, acceleration).

- **Control Module:** This module translates the planned trajectory into specific commands for the hardware actuators, such as steering angle, braking force, and throttle acceleration. The planning module sends the desired trajectory or behavioral commands (speed, acceleration, curvature) to the control module.

1.2 Question 2

Stimulation is the safety check flag that must be passed so we can deploy the autonomous rover in real world to let you fail cheaply and fix things early.

- **Design phase:** You prototype algorithms (perception, planning, control) in simulation first. You check if they even make sense before touching real hardware.
- **Integration phase:** You connect modules (ROS 2 nodes, sensors, controllers) and verify data flow, timing, and interfaces. Simulation exposes mismatches early (wrong topics, bad transforms).
- **Validation phase:** You test system behavior under different scenarios (terrain, noise, obstacles). You verify requirements like “can it reach goal without collision.”
- **Risk reduction:** You simulate dangerous or rare cases (sensor failure, extreme terrain, bad lighting) that are expensive or unsafe in reality.
- **Pre-deployment testing:** You run full system tests repeatedly to ensure stability before real-world deployment, where debugging becomes expensive.

1.3 Question 3

Simulation environments are essential for developing complex systems, yet they inherently differ from the real world due to the "reality gap"—a discrepancy caused by necessary simplifications, modeling assumptions, and computational constraints. These environments provide a static, deterministic representation, whereas the real world is dynamic, stochastic, and continuous. These approximations affect the overall behavior of the simulation regarding physics, sensing, and timing.

- **Physics:** Simulators use shortcuts to save power, like treating objects as perfectly rigid shapes that never bend or wear down, and simplifying friction into a single fixed number; in reality, materials squash, heat up, and grip differently depending on the surface.
- **Sensing/Perception:** Perception creates a gap because simulated cameras and sensors see a world of perfect lighting and clean lines, whereas real-world sensors deal with "noise" like lens flares, dust on the glass, and confusing reflections off shiny floors that create "ghost" objects.
- **Timing:** Timing is a major hurdle because while a simulator moves in digital manner, the real world is a continuous behavior. If a simulator misses a tiny split-second collision between two of its frames, the entire system can become unstable, causing the robot to shake or crash because it is effectively reacting to the "past" rather than the "now."

Essentially, bridging this gap requires making simulations more "purposefully imperfect", so when the system is deployed in real world, it doesn't crash.

1.4 Question 4

An unrealistic sensor model means the simulated sensor provides "perfect" or oversimplified data that does not represent the noise, latency, bias, or environmental limitations of a real-world physical sensor. While Gazebo can produce perfect data by default (e.g., in ray sensors), real-world sensors are subject to physical distortions, making these idealized models a major source of the "reality gap" in robotics.

Characteristics of Unrealistic Models

- **Idealized Noise Models:** The model lacks realistic, complex, or state-dependent noise, relying instead on Gaussian noise or no noise at all. Real sensors may have non-Gaussian noise, quantization errors, or noise dependent on operating conditions (e.g., temperature).
- **Absent or Constant Bias:** The model fails to simulate drift or offset, such as IMU bias, which changes over time.
- **Perfect Update Rate:** The sensor updates instantly or at a perfectly constant rate, failing to simulate varying latency, packet loss, or reduced update rates at high load.
- **Simplified Environmental Interaction:** The sensor treats the environment as ideal (e.g., LiDAR rays pass through glass, or cameras don't experience lens flare/reflections), neglecting complex light scattering, material properties, or multi-path reflections.

Impact on System Performance

Unrealistic sensor models can lead to a robot that performs perfectly in simulation but fails upon deployment.

- **Inaccuracies in Noise/Bias (e.g., IMU/GPS):** Localization algorithms (SLAM, EKF) become overconfident and over-tuned, failing to handle real-world drift or signal losses, so the Rover or robot drifts off path or gets lost when transitioning from simulation to reality.
- **Inaccuracies in Update Rate:** Control loops are tuned for high, stable frequencies, but the latencies and packet losses cause unstable control or failure of time-sensitive navigation algorithms when real-world data arrives with latency or jitter.
- **Environmental Interaction Inaccuracies (e.g., Cameras/LiDAR):** Perception algorithms overfit to perfect synthetic images, missing objects due to glare, dirt, or complex reflections that aren't modeled. Vision-based models (YOLO, object detection) may fail under changing lighting, and LiDAR might fail to detect obstacles with specific material properties.

- **Neglected Physical Interactions (e.g., Contact Sensors):** Frictional or collision-based sensors (like contact sensors) may give unrealistic "stuck" or "uninterrupted" signals. As a result, poor grasping performance in manipulation tasks, or navigation failure on uneven terrain.

2 ROS2

2.1 Question 1

- **Node:** A node is like a class that contains functions and commands performs computation. Such as a node for the camera, a node for motor control, or a node for LIDAR processing. A system is made of multiple nodes working together.
- **Message:** A message is the data structure used to share information between nodes. It defines the type of data sent (e.g., `std_msgs/msg/String` for text, `geometry_msgs/msg/Twist` for velocity).
- **Topic:** A topic is a named communication channel, acting as a bus where nodes exchange messages. Topics act like a radio broadcast: a node can "tune in" (subscribe) or "broadcast" (publish) to a topic. Topics are crucial for continuous data streams like sensor readings. They can be one to many or many to one or many to many.
- **Publisher:** A component within a node that sends data to a topic. It tells the ROS 2 system, "I am going to publish this specific message type to this topic name".
- **Subscriber:** A component within a node that listens to a topic to receive data. It tells the ROS 2 system, "I want to receive messages of this type from this topic name," and defines a callback function to handle incoming data.
- **Launch File:** A file (usually written in Python) that simplifies launching multiple nodes simultaneously, setting parameters, and configuring the system. It allows you to start an entire complex robot system with one command, `ros2 launch`.

How do they work together: Sensors run as nodes, they publish messages (e.g., images, IMU output data) to topics, perception nodes subscribe, they process, and publish results (objects, poses) to localization, so localization subscribes and fuses the data received through a topic so the planning subscribes and outputs a path, then finally control subscribes and sends commands to actuators to actually make the rover act.

2.2 Question 2

Quality of Service (QoS) in ROS 2 defines a set of communication policies that control how data is exchanged between publishers and subscribers. These policies are implemented through the underlying DDS middleware and allow developers to tune communication behavior based on system requirements such as reliability, latency, and data persistence.

- **Reliability:** One of the key QoS policies is reliability. In the **reliable** setting, the system guarantees that all messages are delivered, using retransmissions if necessary. This is suitable for critical data such as control commands or state information (like battery voltage). In contrast, the **best effort** setting does not guarantee delivery and does not attempt retransmission, making it more suitable for high-frequency sensor data where low latency is more important than completeness.
- **Durability:** Another important policy is durability. The **transient local** setting allows a publisher to store the last message and deliver it to any new subscribers that join later. This is useful for data that must be available even if subscribers start after the publisher. The **volatile** setting does not store any messages, meaning subscribers only receive data published after they are connected. This is typically used for real-time data streams.

Compatibility: A mismatch between publisher and subscriber QoS settings can lead to communication failure. For example, if a subscriber requires reliable communication but the publisher uses best effort, the connection may not be established. Similarly, if a subscriber expects transient local durability but the publisher is configured as volatile, the subscriber will not receive previously published data. Therefore, QoS compatibility is essential to ensure proper data exchange within a ROS 2 system.

3 Path Planning

3.1 Question 1

Path planning is the computational process of determining a collision-free, feasible route for an autonomous robot from its current position (or configuration) to a target goal position, while satisfying environmental restrictions and kinematic limitations. It is a core component of robotics that enables autonomy by transforming a high-level goal into an actionable, low-level trajectory.

Role within the Navigation Stack

Path planning is the central intelligence module within a robot's **navigation stack** (often referred to in ROS as `move_base` or `Nav2`). It ensures the robot can navigate complex, often dynamic environments. Its role is divided into two hierarchical levels:

Determining a Feasible Trajectory

A robot determines a feasible path by mapping the environment, understanding its own constraints, and applying search or optimization algorithms.

1. Environmental Constraints (Static and Dynamic)

- **Map Representation:** The robot creates an **occupancy grid map** (2D or 3D) using LIDAR or camera data, where space is divided into free, occupied, or unknown zones.
- **Costmaps & Inflation:** Obstacles are not treated as just points; they are **inflated** by a radius equivalent to the robot's size to create a safety buffer, ensuring the robot doesn't graze walls.
- **Dynamic Obstacles:** Local planners (e.g., DWA) use real-time sensor updates to detect moving objects and recompute trajectories instantly to avoid collisions.

2. Kinematic and Dynamic Constraints

A path is only feasible if the robot can physically follow it.

- **Non-holonomic Constraints:** Robots like cars (Ackermann steering) or differential-drive bots cannot move sideways. Path planners must generate paths that respect these turning radius limitations.
- **Velocity/Acceleration Limits:** Planners calculate paths that do not require abrupt stops or speeds higher than the motor's capabilities.
- **Trajectory Optimization (TEB):** Techniques like Time Elastic Bands (TEB) represent the path as a sequence of poses, solving a non-linear optimization problem to generate smooth trajectories that minimize time and curvature, keeping the path drivable.

3.2 Question 2

Global and local path planners serve different but complementary roles in the navigation stack of an autonomous system.

The **global planner** operates at a high level and computes a complete path from the robot's current pose to the goal using a **global map** (typically an occupancy grid). In the ROAR system, this is implemented using the A* algorithm. It assumes mostly static environments and focuses on **optimality and collision-free path generation**. The output is a reference trajectory that guides the robot over long distances.

In contrast, the **local planner** operates in real time and focuses on short-term motion execution. It uses **local sensor data** (e.g., LiDAR, camera) to react to dynamic or previously unknown obstacles. In the ROAR system, the Artificial Potential Field (APF) method is used, which continuously adjusts velocity commands based on attractive forces toward the goal and repulsive forces from obstacles.

The interaction between both planners is hierarchical. The global planner provides a **reference path**, while the local planner ensures safe execution by **modifying motion commands in real time**.

If deviations occur due to obstacles, the local planner adjusts the trajectory without recomputing the full global path. In some cases, significant deviations may trigger a global re-planning step.

Thus, the global planner ensures **long-term optimality**, while the local planner ensures **real-time safety and adaptability**, making both essential for robust navigation in dynamic environments.

3.3 Q3: Occupancy Grid vs. Height Maps

Occupancy Grid Maps: A 2D or 3D grid where each cell stores the probability of containing an obstacle.

- **Advantages:** Simple to implement, computationally fast for planar navigation, and easy to update.
- **Limitations:** Memory-intensive in 3D; assumes a flat environment.
- **Use Cases:** Indoor mobile robots, flat-surface navigation.

Height Maps: A 2D grid where each cell stores the maximum surface elevation (Z-value) at that (X,Y) coordinate.

- **Advantages:** Memory-efficient representation of uneven 3D terrain (2.5D).
- **Limitations:** Cannot represent multiple elevations at one coordinate (e.g., overhangs, tunnels).
- **Use Cases:** Outdoor rovers, planetary exploration.

3.4 Question 4

A* Algorithm

The A* algorithm is a graph-based search algorithm used to find the lowest-cost path from a start node to a goal node. In autonomous systems, it is commonly used as a global planner operating on an occupancy grid map.

The algorithm evaluates nodes using the cost function:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ is the actual cost from the start node to node n
- $h(n)$ is the heuristic estimate from node n to the goal
- $f(n)$ is the estimated total cost of the path through node n

Algorithm Operation

1. Initialize an open list (nodes to explore) and a closed list (visited nodes)
2. Start from the initial node and compute $f(n)$
3. Select the node with the lowest $f(n)$ from the open list
4. Move it to the closed list and expand its neighbors
5. For each neighbor:
 - Compute $g(n)$
 - Estimate $h(n)$ (e.g., Euclidean distance)
 - Compute $f(n)$ and store the parent node
6. Repeat until the goal node is reached

Heuristic Function

The heuristic $h(n)$ guides the search toward the goal and reduces computation time. Common choices include Euclidean or Manhattan distance.

Optimality Conditions

A* guarantees an optimal path if:

- **Admissibility:** $h(n) \leq h^*(n)$ (never overestimates the true cost)
- **Consistency:** For any node n and its successor n' , the estimated cost of reaching the goal from n must be no greater than the step cost to n' plus the estimated cost from n' . This ensures $f(n)$ values are non-decreasing along any path, preventing the algorithm from having to re-explore nodes.

$$h(n) \leq c(n, n') + h(n')$$

Under these conditions, A* always finds the shortest path without re-expanding nodes.

Application in ROAR System

In the ROAR architecture, A* is used as the global planner. It computes a collision-free path using the occupancy grid map, which is then followed and refined by the local planner during real-time navigation.

3.5 Q5: Artificial Potential Field (APF)

The APF method models the robot moving under the influence of an artificial physical field. The target goal generates an **attractive force** pulling the robot, while obstacles generate **repulsive forces** pushing it away. The robot follows the negative gradient of the total potential field.

- **Limitations:** Prone to getting trapped in **local minima** (where forces cancel out before the goal) and oscillatory behavior in narrow corridors.

4 Control

4.1 Q1: Control Theory

Control theory is the mathematical framework of manipulating system inputs to force a system's output to follow a desired reference. In robotics, its purpose is to govern physical actuators to ensure precise, stable, and responsive execution of trajectories.

4.2 Q2: Open-Loop vs. Closed-Loop

- **Open-Loop:** Computes control inputs based solely on the desired target, without observing the actual output. Cannot correct disturbances.
- **Closed-Loop:** Utilizes sensor feedback to continuously compare actual output against the desired target. The resulting error is used to dynamically adjust inputs and correct deviations.

4.3 Q3: Controller Stability

A stable controller ensures that over time, the system's error converges toward zero (or remains bounded) following a disturbance. An unstable controller causes the system's output to oscillate infinitely or diverge destructively.

4.4 Q4: Pure Pursuit

Pure Pursuit is a geometric path-tracking algorithm. It establishes a "lookahead point" at a fixed distance ahead on the desired path and calculates the curvature of a circular arc connecting the robot's rear axle to this point, translating that curvature into a steering command.

4.5 Q5: Adaptive Pure Pursuit

Standard Pure Pursuit uses a fixed lookahead distance. Adaptive Pure Pursuit dynamically scales the lookahead distance based on the robot's current velocity. Adaptation prevents unstable oscillations at high speeds (requires longer lookahead) and prevents corner-cutting at low speeds (requires shorter lookahead).

5 Localization

5.1 Q1: Sensor Fusion

Sensor fusion is the mathematical combination of data from multiple sensors to produce a single state estimate more accurate and robust than any individual sensor. It mitigates the inherent failure modes of individual sensors (e.g., combining fast IMU updates with drift-free GPS).

5.2 Q2: IMU Function and Application

An Inertial Measurement Unit (IMU) consists of accelerometers and gyroscopes.

- **Data:** Measures linear acceleration (m/s^2) and angular velocity (rad/s).
- **Usage:** Integrated over time to compute changes in velocity, position, and orientation (dead reckoning).
- **Noise and Drift:** Because it mathematically integrates noisy readings, errors accumulate, leading to unbounded positional drift over time.

5.3 Q3: Kalman Filter (KF) vs. Extended Kalman Filter (EKF)

- **Kalman Filter (KF):** An optimal estimator for strictly **linear** systems subject to Gaussian noise.
- **Extended Kalman Filter (EKF):** An adaptation for **non-linear** systems. It linearizes models around the current estimate using partial derivatives (Jacobians).
- **Usage:** Use KF for simple linear dynamics. Use EKF for robotic kinematics, which are inherently non-linear due to trigonometric functions.

6 Perception

6.1 Q1: LiDAR vs. Camera

- **LiDAR:** Emits laser pulses to measure time-of-flight, producing highly precise 3D geometric point clouds. Processing relies on spatial algorithms. Fails in heavy rain, fog, or against reflective/transparent surfaces.
- **Camera:** Captures passive ambient light to produce 2D pixel arrays. Processing relies on computer vision algorithms. Fails in low-light, direct glare, or featureless environments.

6.2 Q2: Monocular vs. Stereo Cameras

- **Monocular:** Single lens. Cannot inherently perceive absolute depth from a single frame without prior knowledge or movement (optical flow). Low computational overhead.
- **Stereo:** Dual lenses with a known baseline. Calculates depth instantly by comparing horizontal pixel shift (disparity) via triangulation. High computational requirement.

6.3 Q3: ArUco Markers

ArUco markers are synthetic, high-contrast square fiducials containing a unique binary matrix. Detected by thresholding an image, finding square contours, and extracting the ID.

- **Extracted Information:** Marker ID and precise 6D pose (3D translation, 3D rotation) relative to the camera.
- **Usage:** Ground-truth landmarks for visual odometry, precise docking, or object manipulation.

7 Conclusion

This solo mission provided a comprehensive transition from theoretical autonomous systems concepts to practical implementation and debugging within a modern ROS 2 (Humble) and Ignition Gazebo environment.

Through the completion of this project, the following core competencies were developed:

- **Architectural Understanding:** Gained a solid foundation in the distributed nature of ROS 2, including the configuration of nodes, topics, and Quality of Service (QoS) policies to ensure reliable inter-process communication.
- **System Modeling & Simulation:** Successfully designed a custom robotic platform from scratch using URDF and Xacro. Learned to integrate Ignition-specific system plugins (DiffDrive, IMU, Camera) and construct a custom simulation world with static models.
- **Sensor Integration & Data Bridging:** Mastered the `ros_gz_bridge` to successfully transport critical simulation data (clock, joint states, odometry, IMU, and raw image feeds) into the ROS 2 ecosystem for downstream processing.
- **Perception & Spatial Reasoning:** Implemented an end-to-end computer vision pipeline using OpenCV to detect ArUco markers. Successfully resolved complex Python dependency conflicts (NumPy/OpenCV mismatches) and utilized camera intrinsic parameters to solve the PnP problem, visualizing 6D spatial poses in RViz2.
- **Debugging & System Verification:** Developed rigorous troubleshooting skills. Navigated and resolved issues ranging from XML syntax errors and missing frame IDs to shared memory conflicts and frame transformations, ultimately validating the system using `rqt_graph` and `tf2_tools`.

Ultimately, this mission highlighted that building an autonomous system is not merely about writing code, but about ensuring deterministic data flow, managing complex dependencies, and bridging the gap between simulated physics and algorithmic perception.